LinkiaFP

PLANTILLA: Entrega de Actividades

6-27_1_1_M-IA_5074_Sistemas de Big Data

MASTER IA & BIG DATA

Curso académico 2026/2027 - Edición Online

Acceso al repositorio de ejercicios para el Módulo actual sobre Sistemas de Big Data
https://github.com/dcueli-com/bigdata_sys.git

Tema 4: Gestión y Almacenamiento de Datos en Big Data

Índice

1. Ejercicio 1
   a. ***Acceso al repositorio para el ejercicio***
   b. *Carga un conjunto de datos grande (un CSV o un archivo de texto) en Python.*
   c. *Realiza algunas operaciones básicas como filtrado, ordenación y agregación de datos utilizando tanto Pandas como Polars.*
   d. *Compara el tiempo de ejecución de las operaciones en ambos casos.*
2. Ejercicio 2
   a. ***Acceso al repositorio para el ejercicio***
   b. Utiliza el framework PySpark.
   c. Crea un DataFrame con las columnas "Nombre", "Edad", "Peso" y "Talla".
   d. Define los tipos de datos de cada columna (StringType, IntegerType).
   e. Introduce información de 10 personas ficticias en el DataFrame.
   f. Aplica un filtro al DataFrame para seleccionar las personas que cumplen con los criterios de peso y talla.
   g. Muestra el resultado del filtrado en la consola
      i. Preparación del entorno de desarrollo
      ii. Análisis de Valores ausentes o Nulos
         ➢ *Resultados del Análisis*
      iii. Análisis de Valores duplicados
         ➢ *Resultados del Análisis*
      iv. Análisis de Integridad en el Cálculo de Totales
         ➢ *Resultados del Análisis*
      v. Análisis de Valores no válidos
         • Análisis de Fechas no válidas
         • Análisis de Valores numéricos no válidos
3. Ejercicio 6
   a. ***Acceso al repositorio para el ejercicio***
   b. Crea un dataset de al menos 10 frases relacionadas con temas técnicos como Big Data, MapReduce, y PySpark
   c. Aplica el modelo MapReduce en PySpark siguiendo estas fases
      i. Map: Divide las frases en palabras y asigna un valor 1 a cada palabra.
      ii. Shuffle: Agrupa todas las palabras iguales.
      iii. Reduce: Suma las ocurrencias de cada palabra.
   d. El resultado debe mostrar la frecuencia de cada palabra en el conjunto de frasestiliza el framework PySpark.

Nota:
Usar el código tal cual puede ser expuesto aquí no funcionaría ya que estaría a falta de las importaciones particulares del desarrollo de la propia práctica como son definiciones, tipos personalizados, variables globales, funciones, módulos, etc; por lo que no ha sido añadido en este informe, aunque sí el acceso al repositorio y algunos archivos individuales del mismo. Igualmente, como se habrá podido comprobar, he adjuntado un comprimido como si de la descarga (sin `pip install`) se tratase.

[Ir al índice]

EJERCICIO 1: Pandas vs Polars:

Acceso al repositorio para el ejercicio

https://github.com/dcueli-com/bigdata_sys/tree/develop/src/activities/RA3/lesson_04

El entorno de desarrollo es ***Debian 13 (Trixie)*** bajo ***WSL*** en ***Windows 11 Pro 25H2*** con el *gestor de versiones* de Python ***pyenv*** y usando la versión 3.13.7 del lenguaje del entorno virtual, aunque como se puede observar en la Figura 0 los distintos kernel de Python están sincronizados para evitar confusiones.

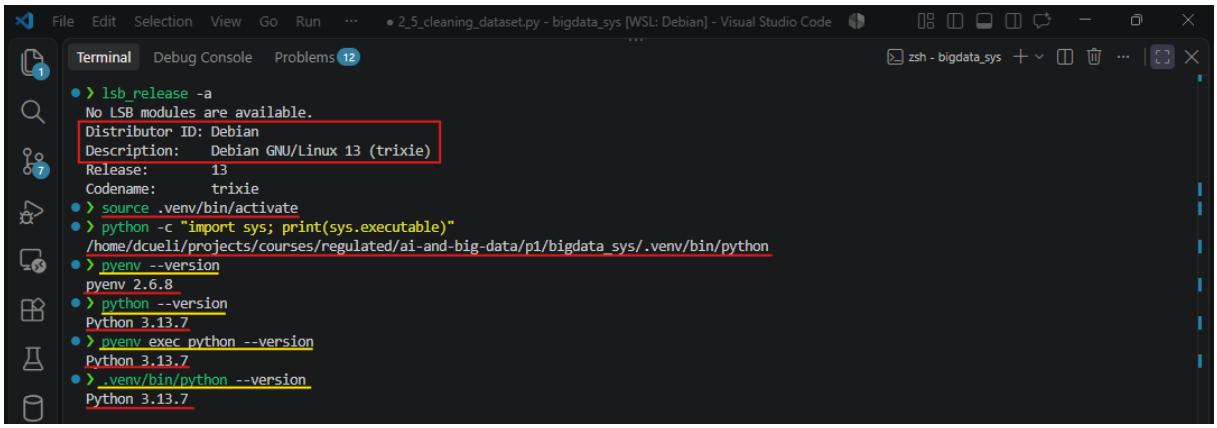


Figura 0.

Instalamos las bibliotecas *Pandas y Polars*

```
pip install pandas polars
```

a. Carga un conjunto de datos grande (un CSV o un archivo de texto) en Python [Ir al índice]

Para este ejercicio he usado el dataset de MovieLens 20M Dataset como hicimos en el módulo de Bigdata Aplicado para los ejercicios obligatorios de RA1

Una vez instalado todo podremos manipular, validar y visualizar los diferentes resultados así que, tanto con Pandas, como con Polars, cargamos los conjuntos de datos:

- *«movies.csv»*, y
- *«ratings.csv»*

Como previamente se conoce las columnas y formato de cada una de las columnas de los conjuntos de datos de *MovieLens*, sabemos que el año de estreno de las películas está en la columna *«Title».* Así que, como *valor añadido*, vamos a hacer la limpieza de datos de conjunto.

Una limpieza completa y con valor añadido de movies.csv consiste en tres operaciones vectorizadas tanto en *Pandas* como en *Polars:*

- *Extraer el año a la nueva columna «year»*
- *Limpiar la columna «title»*
- *Purgar registros sin año válido*

Además de ser lo más lógico y óptimo de cara a posibles análisis y visualización de informes estadísticos, esta limpieza la he desarrollado con la idea de poder hacer filtrados limpios mediante el año de estreno de las películas.

En la siguiente imagen (Figura 1) podemos ver los tiempos de empleados por ambas bibliotecas, observando cómo ***Polars emplea bastante menos tiempo***.

```python
# LIBRARIES COMPARISON, 4.1.1 LOAD SOURCE DATASET (from CSV file)
# ================================================================
tInit: float = time.time()
csl.SectionBegin("LIBRARIES COMPARISON, 4.1.1 LOAD SOURCE DATASET (from CSV file)\n+ Loading Movies & Ratings Datasets", 0)

try:
  # a. PANDAS::LOAD DATASET
  mvDfPd = bdo.BigDataOrchestrator.CleanMoviesData('pandas',
    bdo.BigDataOrchestrator.LoadDataset('pandas', str(mvDataset), pPrintResult=False)['data'], 3
  )['data']
  rtDfPd: pd.DataFrame = bdo.BigDataOrchestrator.LoadDataset('pandas', str(rtDataset))['data']

  # b. POLARS::LOAD DATASET
  mvDfPl = bdo.BigDataOrchestrator.CleanMoviesData('polars',
    bdo.BigDataOrchestrator.LoadDataset('polars', str(mvDataset), pPrintResult=False)['data'], 3
  )['data']
  rtDfPl: pl.DataFrame = bdo.BigDataOrchestrator.LoadDataset('polars', str(rtDataset))['data']

except Exception as e:
  csl.SectionErrEndAndExit(e.args[0], tInit)
```

[41] ✓ 9.0s Python

```
+ ============================================================
+ LIBRARIES COMPARISON, 4.1.1 LOAD SOURCE DATASET (from CSV file)
+ Loading Movies & Ratings Datasets
+ ------------------------------------------------------------
Executing operation with PANDAS...

Movies dataset cleaned ('year' extracted) successfully. Execution time: 0.0617 seconds
Total rows: 27255
First 3 rows of the dataset:
   movieId            title                                        genres  \
0        1        Toy Story  Adventure|Animation|Children|Comedy|Fantasy
1        2          Jumanji                   Adventure|Children|Fantasy
2        3  Grumpier Old Men                               Comedy|Romance

   year
0  1995
1  1995
2  1995
+ ------------------------------------------------------------
Executing operation with PANDAS...

File loaded: /home/dcueli/projects/courses/regulated/ai-and-big-data/p1/bigdata_sys/src/activities/RA3/lesson_04/data/ml-20m/ratings.csv
 successfully. Execution time: 8.6648 seconds
Total rows: 20000263
First 5 rows of the dataset:
   userId  movieId  rating   timestamp
0       1        2     3.5  1112486027
1       1       29     3.5  1112484676
2       1       32     3.5  1112484819
3       1       47     3.5  1112484727
4       1       50     3.5  1112484580
+ ------------------------------------------------------------
Executing operation with POLARS...

Movies dataset cleaned ('year' extracted) successfully. Execution time: 0.0094 seconds
Total rows: 27255
First 3 rows of the dataset:
shape: (3, 4)
┌─────────┬──────────────┬─────────────────────────────┬──────┐
│ movieId ┆ title        ┆ genres                      ┆ year │
│ ---     ┆ ---          ┆ ---                         ┆ ---  │
│ i64     ┆ str          ┆ str                         ┆ i64  │
╞═════════╪══════════════╪═════════════════════════════╪══════╡
│ 1       ┆ Toy Story    ┆ Adventure|Animation|Children|C… ┆ 1995 │
│ 2       ┆ Jumanji      ┆ Adventure|Children|Fantasy  ┆ 1995 │
│ 3       ┆ Grumpier Old Men ┆ Comedy|Romance          ┆ 1995 │
└─────────┴──────────────┴─────────────────────────────┴──────┘
+ ------------------------------------------------------------
Executing operation with POLARS...

File loaded: /home/dcueli/projects/courses/regulated/ai-and-big-data/p1/bigdata_sys/src/activities/RA3/lesson_04/data/ml-20m/ratings.csv
 successfully. Execution time: 0.2432 seconds
Total rows: 20000263
First 5 rows of the dataset:
shape: (5, 4)
┌────────┬─────────┬────────┬───────────┐
│ userId ┆ movieId ┆ rating ┆ timestamp │
│ ---    ┆ ---     ┆ ---    ┆ ---       │
│ i64    ┆ i64     ┆ f64    ┆ i64       │
╞════════╪═════════╪════════╪═══════════╡
│ 1      ┆ 2       ┆ 3.5    ┆ 1112486027 │
│ 1      ┆ 29      ┆ 3.5    ┆ 1112484676 │
│ 1      ┆ 32      ┆ 3.5    ┆ 1112484819 │
│ 1      ┆ 47      ┆ 3.5    ┆ 1112484727 │
│ 1      ┆ 50      ┆ 3.5    ┆ 1112484580 │
└────────┴─────────┴────────┴───────────┘
```

Figura 1.

b. Realiza algunas operaciones básicas como filtrado, ordenación y agregación de datos utilizando tanto Pandas como Polars [Ir al índice]

Como hemos extraído la información del año de las películas, por así decirlo, a una columna extra, ya podemos hacer filtros limpios, sin acciones ni consumo de recursos extra. En la siguiente imagen (Figura 2) podemos ver los tiempos empleados por ambas bibliotecas para el filtro que he querido emplear para este apartado: *Filtrar las películas a partir del año 2000 con una valoración de más de 4 estrellas* —según dice en la documentación de MovieLens—.

```python
# LIBRARIES COMPARISON, 4.1.2 BASIC OPERATIONS: FILTERING
# ======================================================
csl.SectionBegin("LIBRARIES COMPARISON, 4.1.2 FILTERING\n+ Movies (2000 <= year) & Ratings (4 <= rating)", 0)

try:
  # a. PANDAS::FILTER & JOIN
  tInitPd: float = time.perf_counter()
  fltDfPd: pd.DataFrame = pd.merge(
    bdo.BigDataOrchestrator.FilterData('pandas', mvDfPd, 'year', '>=', 2000, pPrintResult=False)['data'],
    bdo.BigDataOrchestrator.FilterData('pandas', rtDfPd, 'rating', '>=', 4, pPrintResult=False)['data'],
    on='movieId', how='inner'
  )

  csl.SectionBreak(0, 0)
  print(f"Executing operation with PANDAS...")
  csl.nbsp()
  print(fltDfPd.head(3))
  csl.nbsp()
  print(f"Filtered successfully. Dataset total rows: {len(fltDfPd)}")
  print(f"Execution time: {time.perf_counter() - tInitPd:.4f} seconds")

  # b. POLARS::FILTER & JOIN
  tInitPl: float = time.perf_counter()
  fltDfPl: pl.DataFrame = bdo.BigDataOrchestrator.FilterData('polars', mvDfPl, 'year', '>=', 2000, pPrintResult=False)['data'].join(
    bdo.BigDataOrchestrator.FilterData('polars', rtDfPl, 'rating', '>=', 4, pPrintResult=False)['data'], on='movieId', how='inner'
  )

  csl.SectionBreak(0, 0)
  print(f"Executing operation with POLARS...")
  csl.nbsp()
  print(fltDfPl.head(3))
  csl.nbsp()
  print(f"Filtered successfully. Dataset total rows: {len(fltDfPl)}")
  print(f"Execution time: {time.perf_counter() - tInitPl:.4f} seconds")

except Exception as e:
  csl.SectionErrEndAndExit(e.args[0], tInit)

csl.SectionEnd(0)
```

Python

```
+ ======================================================
+ LIBRARIES COMPARISON, 4.1.2 FILTERING
+ Movies (2000 <= year) & Ratings (4 <= rating)
+ ------------------------------------------------------
Executing operation with PANDAS...

   movieId      title       genres  year  userId  rating   timestamp
0     2769  Yards, The  Crime|Drama  2000     156     4.0  1046895842
1     2769  Yards, The  Crime|Drama  2000    1358     5.0   973554543
2     2769  Yards, The  Crime|Drama  2000    2196     4.0   996462288

Filtered successfully. Dataset total rows: 2500965
Execution time: 2.0153 seconds
+ ------------------------------------------------------
Executing operation with POLARS...

shape: (3, 7)
┌─────────┬──────────────────────┬──────────────────────┬──────┬────────┬────────┬────────────┐
│ movieId ┆ title                ┆ genres               ┆ year ┆ userId ┆ rating ┆ timestamp  │
│ ---     ┆ ---                  ┆ ---                  ┆ ---  ┆ ---    ┆ ---    ┆ ---        │
│ i64     ┆ str                  ┆ str                  ┆ i64  ┆ i64    ┆ f64    ┆ i64        │
╞═════════╪══════════════════════╪══════════════════════╪══════╪════════╪════════╪════════════╡
│ 3889    ┆ Highlander: Endgame  ┆ Action|Adventure|Fan ┆ 2000 ┆ 1      ┆ 4.0    ┆ 1112486138 │
│         ┆ (Highlande…          ┆ tasy                 ┆      ┆        ┆        ┆            │
│ 3996    ┆ Crouching Tiger,     ┆ Action|Drama|Romance ┆ 2000 ┆ 1      ┆ 4.0    ┆ 1094785727 │
│         ┆ Hidden Dragon…       ┆                      ┆      ┆        ┆        ┆            │
│ 4011    ┆ Snatch               ┆ Comedy|Crime|Thrille ┆ 2000 ┆ 1      ┆ 4.0    ┆ 1112485406 │
│         ┆                      ┆ r                    ┆      ┆        ┆        ┆            │
└─────────┴──────────────────────┴──────────────────────┴──────┴────────┴────────┴────────────┘

Filtered successfully. Dataset total rows: 2500965
Execution time: 0.2764 seconds
+ ======================================================
```

Figura 2.

De nuevo Polars adelanta a Pandas —*quitándole la pegatinas*—, empleando mucho menos tiempo, pero si se observa un poco el resultado, el ojo experto verá rápidamente que aunque los conjuntos de datos resultantes de Pandas y Polars **contienen exactamente los mismos datos** (2.500.965 filas idénticas) la diferencia en el resultado mostrado por `.head(3)` —según he podido documentarme— que observas se debe a que *las operaciones de filtrado y cruce paralelo en motores de Big Data (como Polars o PySpark) no garantizan un orden de salida determinista*.

El siguiente paso es la *ordenación* (según los requisitos de este punto *«filtrado, ordenación y agregación»*) siendo éste, natural en cualquier flujo de datos.

```python
# LIBRARIES COMPARISON, 4.1.2b BASIC OPERATIONS: SORTING
# ======================================================
tInit: float = time.time()
sortCols: list[str] = ['year', 'rating', 'movieId', 'userId']
sortDirs: list[bool] = [False, False, True, True]

csl.SectionBegin( f"LIBRARIES COMPARISON, 4.1.3 SORTING\n+ Sort by: {sortCols} | Ascending: {sortDirs}", 0)

try:

  # a. PANDAS::SORT DATA
  srtDfPd: pd.DataFrame = bdo.BigDataOrchestrator.SortData(
    'pandas',
    fltDfPd,
    sortCols,
    sortDirs,
    pPreviewRows=3
  )['data']

  # b. POLARS::SORT DATA
  srtDfPl: pl.DataFrame = bdo.BigDataOrchestrator.SortData(
    'polars',
    fltDfPl,
    sortCols,
    sortDirs,
    pPreviewRows=3
  )['data']

except Exception as e:
  csl.SectionErrEndAndExit(e.args[0], tInit)

csl.SectionEnd(0)
```

[6] ✓ 1.2s Python

```
+ ======================================================
+ LIBRARIES COMPARISON, 4.1.3 SORTING
+ Sort by: ['year', 'rating', 'movieId', 'userId'] | Ascending: [False, False, True, True]
+ ------------------------------------------------------
Executing operation with PANDAS...

Data sorted by ['year', 'rating', 'movieId', 'userId'] successfully. Execution time: 0.8711 seconds
Total rows: 2500965
First 3 rows of the dataset:
   movieId              title                   genres  year  userId  rating  \
0   113345  Jupiter Ascending  Action|Adventure|Sci-Fi  2015   19089     5.0
1   113345  Jupiter Ascending  Action|Adventure|Sci-Fi  2015   41852     5.0
2   113345  Jupiter Ascending  Action|Adventure|Sci-Fi  2015   98139     5.0

    timestamp
0  1425219438
1  1424739035
2  1427469523
+ ------------------------------------------------------
Executing operation with POLARS...

Data sorted by ['year', 'rating', 'movieId', 'userId'] successfully. Execution time: 0.3478 seconds
Total rows: 2500965
First 3 rows of the dataset:
shape: (3, 7)
┌─────────┬─────────────────┬─────────────────────────┬──────┬────────┬────────┬────────────┐
│ movieId ┆ title           ┆ genres                  ┆ year ┆ userId ┆ rating ┆ timestamp  │
│ ---     ┆ ---             ┆ ---                     ┆ ---  ┆ ---    ┆ ---    ┆ ---        │
│ i64     ┆ str             ┆ str                     ┆ i64  ┆ i64    ┆ f64    ┆ i64        │
╞═════════╪═════════════════╪═════════════════════════╪══════╪════════╪════════╪════════════╡
│ 113345  ┆ Jupiter Ascending ┆ Action|Adventure|Sci-Fi ┆ 2015 ┆ 19089  ┆ 5.0    ┆ 1425219438 │
│ 113345  ┆ Jupiter Ascending ┆ Action|Adventure|Sci-Fi ┆ 2015 ┆ 41852  ┆ 5.0    ┆ 1424739035 │
│ 113345  ┆ Jupiter Ascending ┆ Action|Adventure|Sci-Fi ┆ 2015 ┆ 98139  ┆ 5.0    ┆ 1427469523 │
└─────────┴─────────────────┴─────────────────────────┴──────┴────────┴────────┴────────────┘
+ ======================================================
```

Figura 3.

Ahora sí podemos ver por el resultado de la imagen anterior (Figura 3) que sí tenemos los mismos resultados y que Polars sigue a la cabeza aunque ahora no por mucho —y que Jupiter Ascending tenga un valoración de 5 estrellas indica que el dataset de MovieLens sí que necesita limpieza—.

Quedaría únicamente el punto de agregación de datos para cerrar este ejercicio.

```python
# LIBRARIES COMPARISON, 4.1.2c BASIC OPERATIONS: AGGREGATION
# ==========================================================
tInit: float = time.time()
groupCols: list[str] = ['year', 'movieId', 'title']
targetCol: str = 'rating'
aggOperation: str = 'mean'
aliasCol: str = 'avg_rating'

csl.SectionBegin(f"LIBRARIES COMPARISON, 4.1.2c AGGREGATION\n+ Group by: {groupCols}\n+ Operation: {aggOperation}('{targetCol}') as '{alia

try:
  # a. PANDAS::AGGREGATE DATA
  aggDfPd: pd.DataFrame = bdo.BigDataOrchestrator.AggregateData(
    'pandas',
    srtDfPd,
    groupCols,
    targetCol,
    aggOperation,
    aliasCol,
    pPreviewRows=3
  )['data']

  # b. POLARS::AGGREGATE DATA
  aggDfPl: pl.DataFrame = bdo.BigDataOrchestrator.AggregateData(
    'polars',
    srtDfPl,
    groupCols,
    targetCol,
    aggOperation,
    aliasCol,
    pPreviewRows=3
  )['data']

except Exception as e:
  csl.SectionErrEndAndExit(e.args[0], tInit)

csl.SectionEnd(0)
```

[10] ✓ 0.5s Python

```
+ ==========================================================
+ LIBRARIES COMPARISON, 4.1.2c AGGREGATION
+ Group by: ['year', 'movieId', 'title']
+ Operation: mean('rating') as 'avg_rating'
+ ----------------------------------------------------------
Executing operation with PANDAS...

Data aggregated by ['year', 'movieId', 'title'] (mean of 'rating') successfully. Execution time: 0.3579 seconds
Total rows: 9735
First 3 rows of the dataset:
   year  movieId        title  avg_rating
0  2000     2769   Yards, The    4.218750
1  2000     3177  Next Friday    4.247292
2  2000     3190    Supernova    4.153409
+ ----------------------------------------------------------
Executing operation with POLARS...

Data aggregated by ['year', 'movieId', 'title'] (mean of 'rating') successfully. Execution time: 0.1598 seconds
Total rows: 9735
First 3 rows of the dataset:
shape: (3, 4)
┌──────┬─────────┬─────────────┬────────────┐
│ year ┆ movieId ┆ title       ┆ avg_rating │
│ ---  ┆ ---     ┆ ---         ┆ ---        │
│ i64  ┆ i64     ┆ str         ┆ f64        │
╞══════╪═════════╪═════════════╪════════════╡
│ 2000 ┆ 2769    ┆ Yards, The  ┆ 4.21875    │
│ 2000 ┆ 3177    ┆ Next Friday ┆ 4.247292   │
│ 2000 ┆ 3190    ┆ Supernova   ┆ 4.153409   │
└──────┴─────────┴─────────────┴────────────┘

+ ==========================================================
```

Figura 4.

Como vemos, Polars completa la agrupación por las columnas *«year»*, *«movieId»* y *«title»*, y el cálculo de la media casi en la mitad de tiempo. Al igual que antes la diferencia no es tan abismal como en el filtrado+join porque los datos ya vienen preordenados del paso anterior, lo que facilita a Pandas la agrupación secuencial en memoria.

c. Compara el tiempo de ejecución de las operaciones en ambos casos

[Ir al índice]

A lo largo de las operaciones realizadas sobre el conjunto de datos de *MovieLens «movies.csv»* y *«ratings.csv»* —claramente con más millones de registros—, los tiempos registrados en cada apartado del ejercicio muestran la misma ventaja de **Polars** frente a **Pandas**.

Aunque ambas bibliotecas funcionan bastante bien en cuanto a millones de registros, leyendo un poco más sobre las diferencias entre ambas, parece ser que las diferencias de tiempo vienen principalmente definidas porque **Pandas** ejecuta la mayoría de sus operaciones en un único hilo de CPU y realiza copias intermedias de memoria. **Polars** por su parte, además de estar escrito en **Rust**, reparte el trabajo entre todos los núcleos del procesador y evitar copias innecesarias de datos en la memoria RAM —lo de «*copias innecesarias*» parece más una mala publicidad hecha y dicha por la competencia, en este caso de Polars contra Pandas, pero lo digo porque así aparece en más de un resultado de comparativas—.

EJERCICIO 2: Crea una base de datos en PySpark que contenga información de 10 personas, incluyendo su nombre, edad, peso y talla. Luego, filtra la base de datos para encontrar a las personas que tienen un peso superior a 70 kg y una talla inferior a 180 cm.

[Ir al índice]

Requisitos:

1. ***Utiliza el framework PySpark.***
2. ***Crea un DataFrame con las columnas "Nombre", "Edad", "Peso" y "Talla".***
3. ***Define los tipos de datos de cada columna (StringType, IntegerType).***
4. ***Introduce información de 10 personas ficticias en el DataFrame.***
5. ***Aplica un filtro al DataFrame para seleccionar las personas que cumplen con los criterios de peso y talla.***
6. ***Muestra el resultado del filtrado en la consola***

Carga el archivo CSV y realiza una exploración inicial de los datos *[Ir al índice]*

Acceso al repositorio para el ejercicio

https://github.com/dcueli-com/bigdata_sys/blob/develop/src/activities/RA1/lesson_02/notebooks/2_5_1_analysing_dataset.ipynb
https://github.com/dcueli-com/bigdata_sys/blob/develop/src/activities/RA1/lesson_02/scripts/2_5_1_analysing_dataset.py

Bueno, como soy de la vieja escuela, he abierto el CSV con Excel, formateado y por supuesto creado el filtro. Una vez hecho esto, le he echado una visual, y se observan los **315 registros** que contiene el archivo además de la primera fila como cabecera de las 5 columnas: *fecha*, *producto*, *cantidad*, *precio_unitario* y *total*; y como no podía ser de otro modo, de mi primer análisis exploratorio mediante juicio experto, se desprende que el dataset presenta múltiples fallos sintácticos, semánticos y de integridad estructural, a mi entender bastante severos, como son: *valores nulos o faltantes*, *fechas inválidas y/o erróneas*, *valores negativos o cero* en las columnas numéricas, *outliers o valores atípicos*, y más casos que seguramente aparecerán cuando le haga una auditoría automatizada.

Preparación del entorno de desarrollo *[Ir al índice]*

Lo primero será ejecutar un análisis completo y para ello vamos a instalar en el entorno las distintas herramientas que sugiere el enunciado de la práctica: *pandas*, *numpy*, *matplotlib*, *seaborn*.

El entorno de desarrollo es ***Debian 13 (Trixie)*** bajo ***WSL*** en ***Windows 11 Pro 25H2*** con el *gestor de versiones* de Python ***pyenv*** y usando la versión 3.13.7 del lenguaje del entorno virtual, aunque como se puede observar en la Figura 0 los distintos kernel de Python están sincronizados para evitar confusiones.

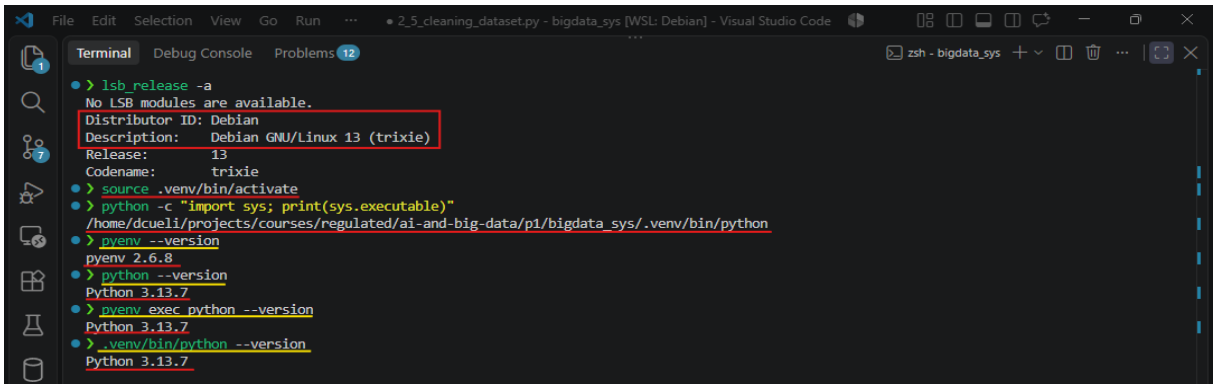


Figura 0.

Instalamos las bibliotecas *Pandas*, *Numpy*, *Matplotlib* y *Seaborn*.

```
pip install pandas numpy matplotlib seaborn
```

Una vez instalado todo podremos manipular, validar y visualizar los diferentes resultados así que cargamos el conjunto de datos del archivo CSV

```
dataset: str = DATA_DIR / "ventas_con_problemas.csv"
df = pd.read_csv(dataset)
```

Identifica y maneja los problemas de calidad de datos *[Ir al índice]*

Análisis de Valores ausentes o Nulos

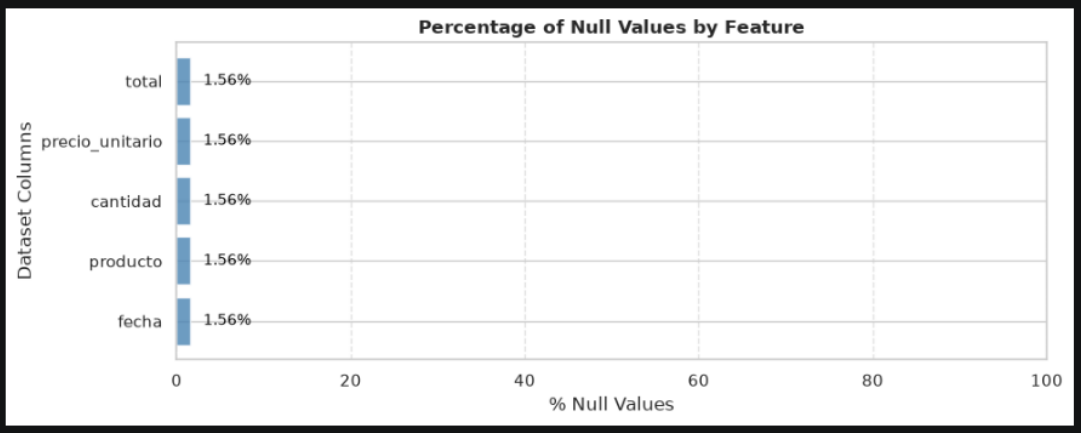
Vamos a inspeccionar exactamente qué columnas tienen valores nulos y cuántos registros están afectados en cada una. Para ello, ejecutamos el código relacionado en la celda con título:

```
# DATA ANALYSIS, 1. CHECK OF NULL VALUES
# ======================================
```

Obteniendo la siguiente información que se puede observar en la Figura 1:

```
# DATA ANALYSIS, 1. CHECK OF NULL VALUES ···

+ ======================================================
+ CHECK OF NULL VALUES
+ ------------------------------------------------------
Null values per column:
fecha             5
producto          5
cantidad          5
precio_unitario   5
total             5
dtype: int64
+ ------------------------------------------------------
Total rows with at least one null value:
25
+ ------------------------------------------------------
```




```
+ ------------------------------------------------------
+ Time to load: 0.2345 seconds
+ ======================================================
```

Figura 1.

Resultados del Análisis de ausentes (o nulos) *[Ir al índice]*
Como análisis final, se han encontrado **5 valores nulos** en cada una de las 5 columnas, lo que corresponde a 5 filas con nulos totales o vacías en esos campos. Estos datos incompletos pueden dificultar el análisis de las ventas y deben revisarse antes de utilizar el conjunto de datos para elaborar informes o tomar decisiones.

En total, hay **25 incidencias de ausentes o nulos** repartidas en **25 filas**, lo que representa el **7,81% de** las **320 filas** del conjunto de datos, que deben ser tratadas, bien eliminándolas por ser registros incompletos críticos, o bien aplicándolas sólo si así se indica en la lógica de negocio.

La recomendación en estos casos es comprobar si los valores ausentes o nulos pueden recuperarse a partir de otras fuentes o si es necesario excluir los registros afectados.

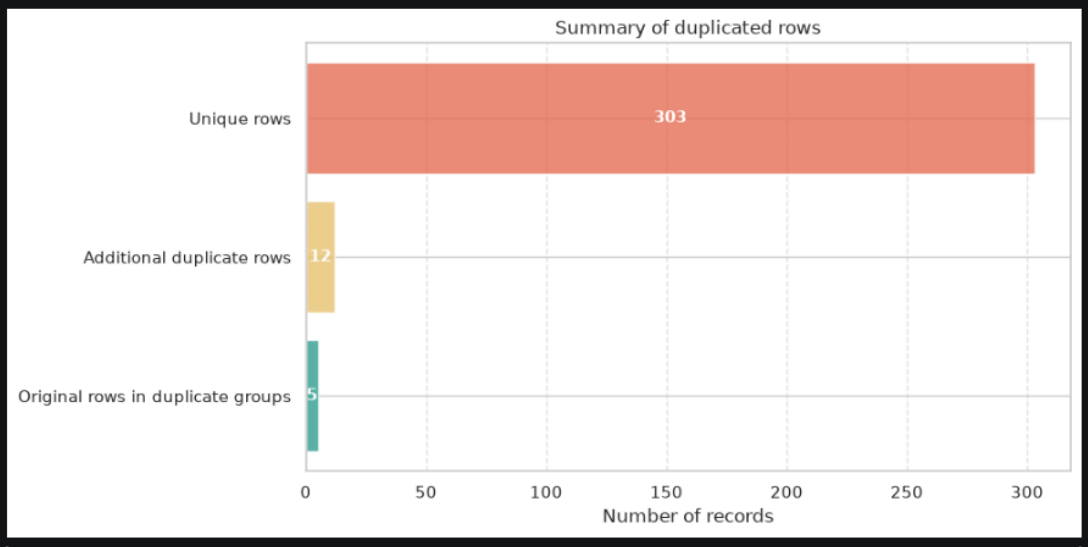
Análisis de Valores duplicados *[Ir al índice]*
Usando el método `.duplicated()` de la biblioteca Pandas, buscamos filas enteras que sean **exactamente iguales** a una anterior, por lo que vamos a usarla indicando el parámetro `keep=False`, que nos marcará todas las repeticiones de las filas duplicadas para poder inspeccionarlas a fondo.
Por otra parte, también podemos contar cuántas filas están repetidas en total.
Ejecutamos el script de la celda:

```
# DATA ANALYSIS, 2. CHECK OF DUPLICATED VALUES
# ============================================
```

Teniendo la salida:

```
# DATA ANALYSIS, 2. CHECK OF DUPLICATED VALUES ⋯
+ ============================================================
+ CHECK OF DUPLICATED VALUES
+ Summary of fully duplicate rows found
+ ------------------------------------------------------------
Just duplicated rows: 12
Source duplicated rows : 5
+ ------------------------------------------------------------
Snapshot duplicated rows: 17

          fecha            producto  cantidad  precio_unitario   total
117  2026-09-24        Funda tablet       2.0            11.14   22.28
315  2026-09-24        Funda tablet       2.0            11.14   22.28
116  2026-10-08    Teclado mecanico       2.0           103.16  206.32
282  2026-10-08    Teclado mecanico       2.0           103.16  206.32
301  2026-10-08    Teclado mecanico       2.0           103.16  206.32
119  2026-10-10       Disco SSD 1TB       2.0           103.23  206.46
175  2026-10-10       Disco SSD 1TB       2.0           103.23  206.46
239  2026-10-10       Disco SSD 1TB       2.0           103.23  206.46
118  2026-10-12  Monitor 24 pulgadas       4.0           127.06  508.24
148  2026-10-12  Monitor 24 pulgadas       4.0           127.06  508.24
217  2026-10-12  Monitor 24 pulgadas       4.0           127.06  508.24
265  2026-10-12  Monitor 24 pulgadas       4.0           127.06  508.24
120  2026-10-17           Webcam HD       2.0            68.97  137.94
122  2026-10-17           Webcam HD       2.0            68.97  137.94
192  2026-10-17           Webcam HD       2.0            68.97  137.94
318  2026-10-17           Webcam HD       2.0            68.97  137.94
319  2026-10-17           Webcam HD       2.0            68.97  137.94
```




```
+ ------------------------------------------------------------
+ Time to load: 0.2200 seconds
+ ============================================================
```

Figura 2.

De la que podemos obtener las siguientes conclusiones para el tratamiento de los datos.

Resultado del Análisis de Valores duplicados *[Ir al índice]*
Para este resultado, se ha optado por un gráfico de barras con el número de registros según su situación de duplicidad. Se muestran los recuentos absolutos, ya que interesa saber cuántas filas están afectadas.
Con los datos que hemos obtenido, claramente la interpretación es:

TIPO DE REGISTRO	FILAS
Filas sin duplicidad (únicas)	303
Filas originales pertenecientes a grupos duplicados	***5***
Filas duplicadas adicionales	***12***
TOTAL	320

Esta distribución presupone que «*las filas originales pertenecientes a grupos duplicados:* ***5***» son las filas originales y que las 12 filas duplicadas son las copias adicionales. Dicho de otro modo, se han detectado exactamente **12 registros** completamente duplicados en columnas clave (*fecha*, *producto*, *cantidad*, *precio_unitario* y *total*).
La ventaja de esta representación es que diferencia dos conceptos que pueden confundirse:

- *12 duplicados adicionales son las filas repetidas que sobran si conservamos una aparición de cada registro.*
- 17 filas afectadas son la suma de las 5 apariciones originales más las 12 repeticiones.

Así, el gráfico permite ver de un vistazo que hay 17 filas implicadas en duplicidades, pero que eliminar únicamente las repeticiones supondría retirar 12 filas, no 17.

La duplicidad de estos registros puede ser debido a fallos en la captura de datos como reintentos fallidos de peticiones HTTP de envíos de formularios o incluso solapamientos al exportar los datos desde la base de datos origen.

Dejar estos 12 registros en el dataset final inflaría artificialmente las estadísticas en cuanto a facturación, las unidades vendidas, etc; lo que se deduce que suprimir estas duplicidades sería lo más apropiado de cara a futuros análisis e informes.

Análisis de Integridad en el Cálculo del Totales *[Ir al índice]*

En un registro correcto, la multiplicación de la `cantidad` por el `precio_unitario` debe ser igual al total grabado en la celda y como en el análisis previo se han observado valores nulos o faltantes, vamos a verificar qué volumen de inconsistencias tiene el dataset en cuanto a la columna `total` se refiere.
Los pasos que llevaremos a cabo serán:

- Calculamos una nueva columna temporal `total_temp`, aquí es importante decir que no tendré en cuenta el signo:

$$total_tmp = cantidad \times precio_unitario$$

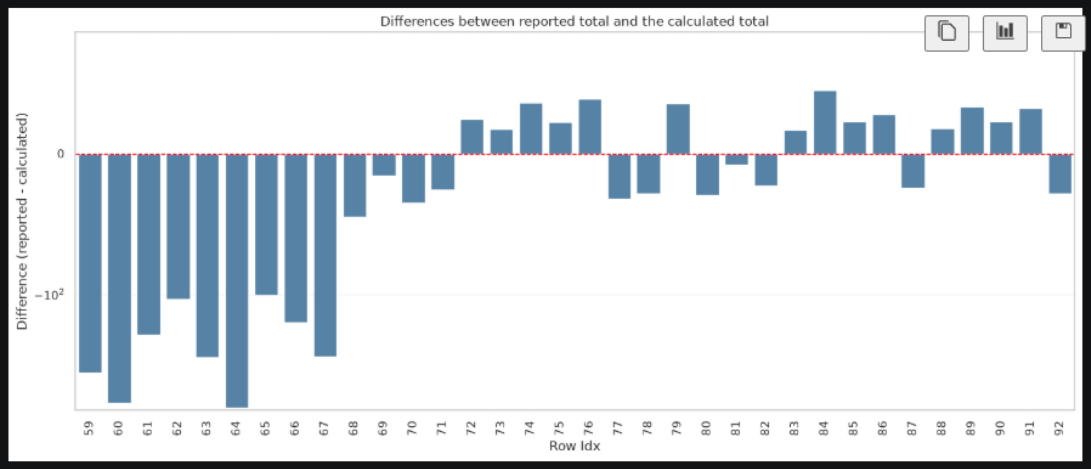
- Comparamos `total_temp` con la columna `total` existente.
- Contamos cuántas filas no coinciden, y para esto es el no tener en cuenta el signo

Ejecutamos el script de la celda:

```
# DATA ANALYSIS, 3. CHECK OF COLUMN VALUE TOTAL
# ============================================
```

Una vez que ejecutado el script, obtenemos la siguiente salida de la Figura 3:

```
+ ============================================================
+ CHECK OF TOTAL COLUMN VALUE
+ Summary of rows with incorrect Total amount:
+ ------------------------------------------------------------
Rows with different totals: 34
+ ------------------------------------------------------------
Snapshot of rows with inconsistent total calculation:
            producto  cantidad  precio_unitario   total  total_temp
59   Teclado mecanico       3.0            69.24 -207.72      207.72
60      Disco SSD 1TB       3.0           120.63 -361.89      361.89
61       Funda tablet       3.0            34.92 -104.76      104.76
62   Altavoz portatil       1.0            54.69  -54.69       54.69
63   Altavoz portatil       4.0            39.26 -157.04      157.04
64   Teclado mecanico       4.0            98.12 -392.48      392.48
65           Hub USB        3.0            16.89  -50.67       50.67
66  Raton inalambrico       2.0            41.59  -83.18       83.18
67   Teclado mecanico       3.0            52.02 -156.06      156.06
68   Teclado mecanico       1.0           107.74   62.83      107.74
```




```
Note:
-----
The chart shows the differences between the reported total in
the CSV and the calculated total obtained by multiplying the
quantity by the unit price for each product. Negative values
indicate that the reported total is lower than the calculated
total, while positive values indicate that it is higher; a
symmetric logarithmic scale is used on the Y-axis to make
differences of varying magnitudes easier to visualize.
+ ------------------------------------------------------------
+ Time to load: 0.6204 seconds
+ ============================================================
```

Figura 3.

`Nota:`
El gráfico representa las diferencias entre el total registrado en el CSV y el total calculado a partir de la cantidad y el precio unitario de cada producto. Los valores negativos indican que el total registrado es inferior al calculado, y los positivos, que es superior; se utiliza una escala simétrica logarítmica en el eje Y para facilitar la visualización de diferencias de distinta magnitud.

Resultados del Análisis de Integridad en el Cálculo de Totales *[Ir al índice]*

Vemos **34 filas con inconsistencias en el cálculo del total**.
Como se observa en la instantánea de la Figura 3, en 10 filas se encuentran dos tipos de errores muy claros que alimentan las 34 discrepancias entre el *total calculado* (`total_temp`) y el *total registrado* (`total`):

1. **Fallo de integridad en los signos negativos (filas 59 a 67)**
   Ya en el primer registro de la instantánea, la cantidad es positiva (`3`) y el precio unitario es positivo (`69.24`), pero el *total registrado* (`total`) aparece en negativo (`-207.72`) cuando debería ser positivo (`207.72`).
2. **Discrepancia en los cálculos puros (fila 68)**
   Un teclado mecánico con cantidad (`1`) y precio unitario (`precio_unitario`) registrado de (`107.74`) da un total esperado de (`107.74`), pero en la columna `total` figura erróneamente otro importe (`62.83`).

Los valores incorrectos deben corregirse tras verificar su origen, evitando sustituirlos automáticamente si no se puede confirmar cuál es el dato correcto. Por tanto, *se recomienda revisar los 34 registros afectados y recalcular el importe a partir de la cantidad y el precio unitario, siempre que estos datos sean fiables.*

Análisis de Valores no válidos [Ir al índice]

Aunque a simple vista puedan parecer que son dos análisis distintos, yo he decidido unificarlos en el mismo gráfico, aunque cada uno tenga a su vez su propio análisis.

Análisis de Fechas no válidas [Ir al índice]

Como a primera vista, he detectado fechas con formato inválido, nulos, ausentes o valores que no son de tipo fecha, voy a analizaros primero.

Estas situaciones suelen ocurrir cuando en los aplicativos, la introducción en los sistemas es libre, es decir, la entrada en el cuadro de texto es abierta y se permite escribir cualquier cosa y, además, en la definición de la columna en la BDD se indica un tipo igualmente abierto, como por ejemplo de tipo texto, véase `VARCHAR(12)`, o `String`, o `Text` en No-SQL, que ni que decir tiene que, normalmente, es un grave error. Si a eso le sumamos la capacidad digital de un orangután que tienen los usuarios, podemos ver escrita cualquier cosa en las celdas de fecha.

En este conjunto de datos en concreto, nos encontramos con varios tipos de errores graves en la columna fecha:

1. **Texto literal inválido**
   Hay filas donde en lugar de una fecha aparecen palabras como "ayer" o "no_es_fecha". Ningún lenguaje puede convertir un texto así en una fecha real.
2. **Fechas con Meses o días fuera de rango**
   - Por ejemplo, aparece `2026-02-30`, `31/02/2026`, o `2026-10-45`.
   - Meses que no existen, como por ejemplo `2026-13-10`.
3. **Celdas vacías (valores nulos)**
   Filas donde directamente no hay fecha registrada.

Para limpiar las fechas de forma segura, utilizamos la función `to_datetime()` de la biblioteca Pandas. Esta función lee el valor e intenta convertirlo a un formato de fecha real (YYYY-MM-DD). Si encuentra un texto extraño, que no puede convertir como un valor nulo, una fecha fuera de rango, o texto puro “`no_es_fecha`”, se le indica que **no dé error**, sino que lo convierta en un valor **`NaN`** para poder detectarlo y eliminarlo, o corregirlo más adelante.

Ejecutamos la celda:

```
# DATA ANALYSIS, 4. CHECK OF INVALID DATE VALUES
# ==============================================
```

Y la ejecución del script da la siguiente salida:

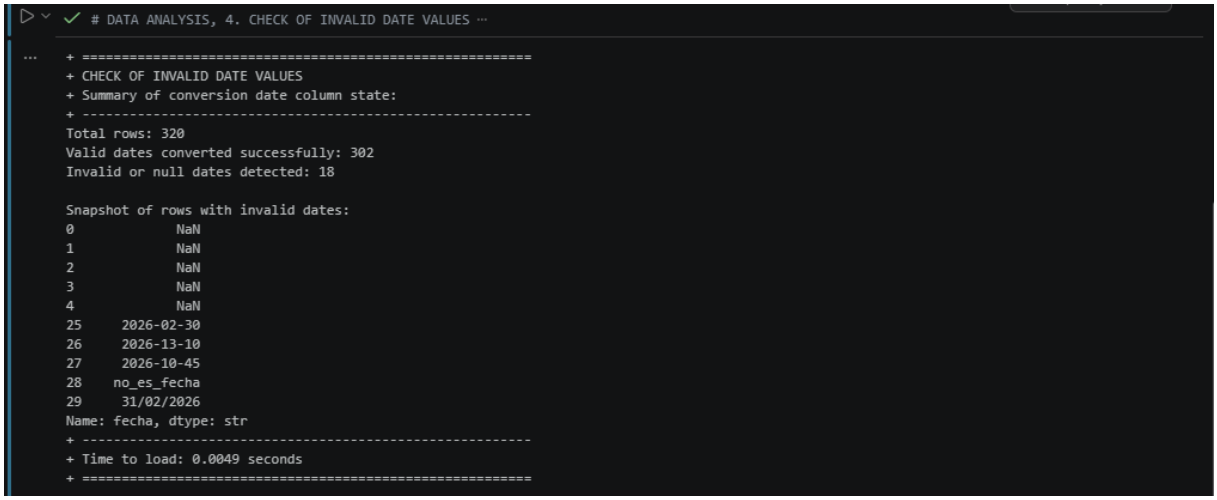


Figura 4.

Análisis de Valores numéricos no válidos *[Ir al índice]*
Una vez ya comprobados los valores nulos (*25 filas afectadas*) y las fechas corruptas (*13 filas con fechas no válidas*), el siguiente problema a revisar en la calidad de los datos son los **valores negativos o cero** en los campos numéricos (`cantidad`, `precio_unitario` y `total`), los cuales no tienen sentido en un registro comercial de movimientos de salida o, dicho de otro modo, más comúnmente movimientos de ventas.

Si entendemos que el dataset aportado procede de un sistema de comercio electrónico, no pueden o no deberían existir precios unitarios negativos, ni totales iguales a cero. Por otra parte, en cuanto a importe de ventas, es decir, a totales se refiere, pueden existir perfectamente con cantidades negativas, pero puede suceder incluso que existan errores de signo.

La solución a importes negativos pasaría por consultar con el dpto. comercial, ventas o administrativo que, normalmente son las entidades dentro de la compañía a las que compete estos asuntos. Un importe negativo representa una devolución siempre y cuando exista un documento de salida (facturas de ventas) con una Serie de numeración explícita y exclusiva para ello. Por tanto, puede ser que existan registros inconsistentes o devoluciones mal registradas sin documento rectificativo asociado, así que se descartaría del dataset para no distorsionar las métricas.

De otra forma, aunque no es el caso, podríamos actuar asumiendo que un registro es rectificativo si, y sólo si, cumple los siguientes requisitos:

1. que en el dataset apareciese la columna cliente,
2. hay un único movimiento para el mismo cliente con la misma cantidad, mismo producto, mismo precio unitario y, por estos dos últimos, el mismo importe (total) en positivo y,
3. según fecha del movimiento, sea anterior al negativo.

Para dar por zanjado este tema, en estas situaciones estaríamos obligados a consultar a las personas y/o departamentos relacionados con la extracción del dataset, o bien, saber que en un dataset nunca, o al menos, nunca deberían estar mezclados las ventas ordinarias y las devoluciones en la misma tabla con signos positivos y negativos ya que podría dar lugar a distorsiones de las métricas de rendimiento de producto, como ya comenté anteriormente.

Con todo esto, ya he dado una pista de la solución que voy a aplicar.

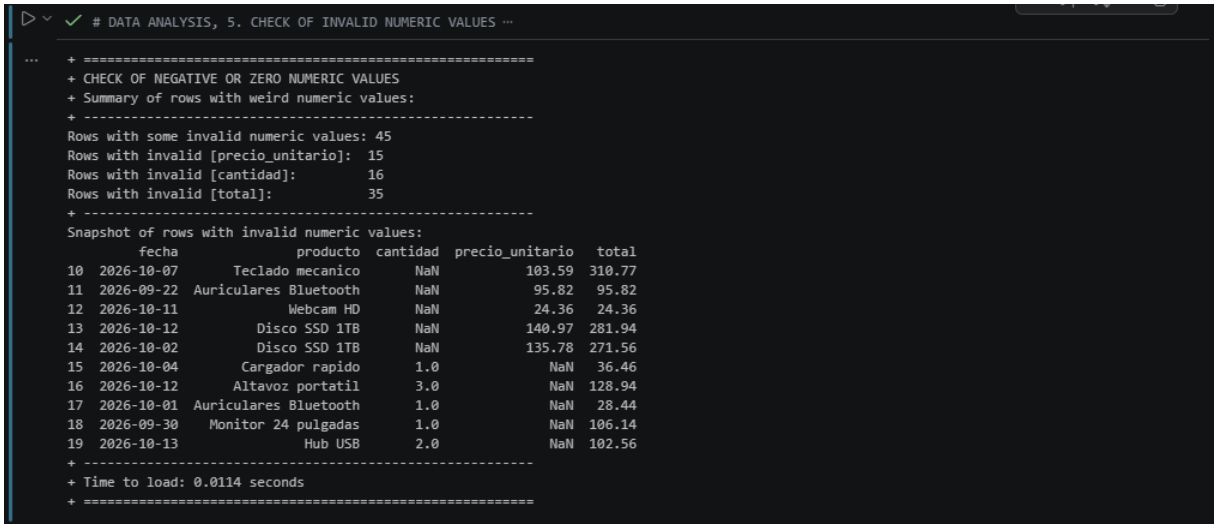
Una vez aclarados los términos, el script para verificar los valores numéricos debe:

- Convertir las columnas `cantidad`, `precio_unitario` y `total` a tipo numérico de forma segura.
- Buscar y mostrar cuántas filas tienen valores menores o iguales a cero (≤ 0) en cada una de esas 3 columnas.
- Mostrar algunos ejemplos de esas filas para ver el problema de cerca.

Ejecutamos el script de la siguiente celda:

```
# DATA ANALYSIS, 5. CHECK OF INVALID NUMERIC VALUES
# ================================================
```

Podemos ver la salida en la siguiente captura de pantalla:



```
# DATA ANALYSIS, 5. CHECK OF INVALID NUMERIC VALUES ⋯
+ ========================================================
+ CHECK OF NEGATIVE OR ZERO NUMERIC VALUES
+ Summary of rows with weird numeric values:
+ --------------------------------------------------------
Rows with some invalid numeric values: 45
Rows with invalid [precio_unitario]:  15
Rows with invalid [cantidad]:         16
Rows with invalid [total]:            35
+ --------------------------------------------------------
Snapshot of rows with invalid numeric values:
         fecha               producto  cantidad  precio_unitario   total
10  2026-10-07       Teclado mecanico       NaN           103.59  310.77
11  2026-09-22  Auriculares Bluetooth       NaN            95.82   95.82
12  2026-10-11             Webcam HD       NaN            24.36   24.36
13  2026-10-12          Disco SSD 1TB       NaN           140.97  281.94
14  2026-10-02          Disco SSD 1TB       NaN           135.78  271.56
15  2026-10-04        Cargador rapido       1.0              NaN   36.46
16  2026-10-12       Altavoz portatil       3.0              NaN  128.94
17  2026-10-01  Auriculares Bluetooth       1.0              NaN   28.44
18  2026-09-30    Monitor 24 pulgadas       1.0              NaN  106.14
19  2026-10-13                Hub USB       2.0              NaN  102.56
+ --------------------------------------------------------
+ Time to load: 0.0114 seconds
+ ========================================================
```

Figura 5.

Resultados de los Análisis *[Ir al índice]*

Resultados del Análisis de fechas no válidas

La salida que vemos en la Figura 3 confirma al 100% lo que se ha comentado en el análisis previo y se puede ver claramente y de forma limpia los ***5 registros vacíos iniciales (***`NaN`***)*** y *los* ***13 errores de fecha*** (`2026-02-30`, `2026-13-10`, `no_es_fecha`, `31/02/2026`, etc.), sumando exactamente las **18 incidencias**.

Resultados del Análisis de Valores numéricos no válidos

Por la Figura 5 se pueden distinguir **45 registros con al menos un valor numérico no válido**. La columna **`total`** es la que presenta más incidencias, **con 35 registros afectados**, seguida de **`cantidad`** con **16** y **`precio_unitario`** con **15**. Cabe destacar que estas cifras pueden solaparse, ya que un mismo registro puede contener errores en varias columnas, por este motivo, no deben sumarse para calcular el total de registros afectados.

En la gráfica que se ha desarrollado, Figura 6, se muestra el porcentaje de *registros correctos* y *con errores* en cada columna (`fecha`, `cantidad`, `precio_unitario` y `total`), lo que permite comparar la calidad de los datos independientemente del número de incidencias detectadas en cada columna. Se observa que la columna total concentra la mayor proporción de registros con valores no válidos.

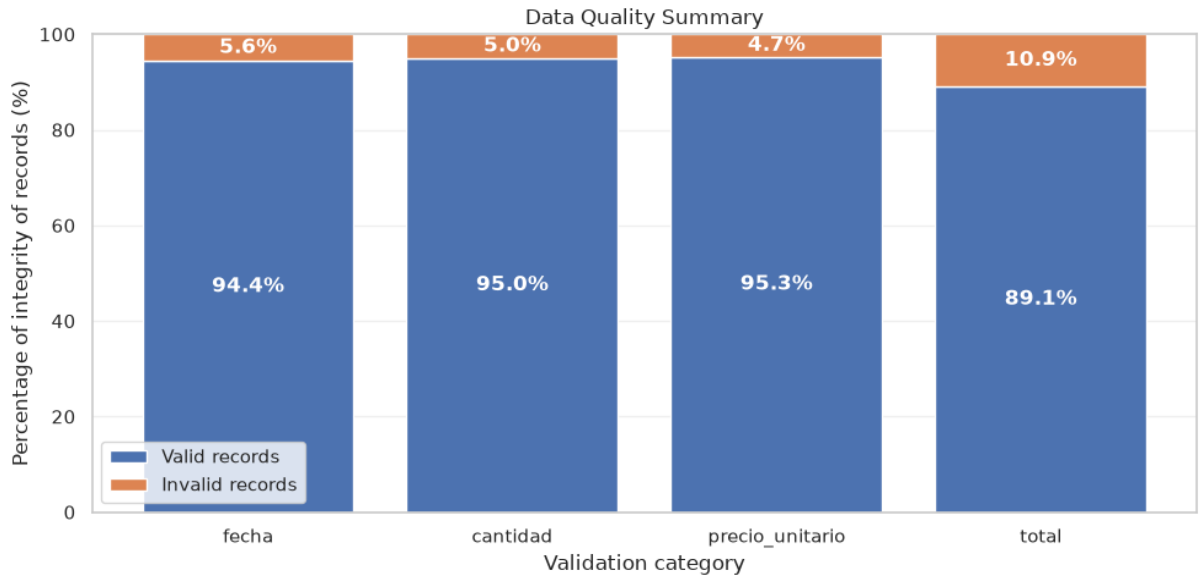


Figura 6.

Se recomienda, primero, tener en cuenta la recomendación del apartado de Resultados del Análisis de Integridad en el Cálculo de Totales para seguidamente revisar los valores ausentes, nulos, cero o negativos. Los valores solo deben corregirse a partir de fuentes fiables antes de utilizar estos datos para los análisis posteriores. Si no es posible recuperarlos, los registros afectados deberían ser excluidos de los análisis que dependan de esos datos.

Análisis de Valores atípicos (Outliers) [Ir al índice]

Sabiendo que los *outliers* son valores que se escapan drásticamente de lo que son los rangos normales de un dato en concreto, y estando en un análisis de datos de comercio electrónico, como en el que más o menos nos encontramos, voy a considerar valores atípicos los casos donde, por ejemplo, un producto como un Cargador o un Ratón, aunque sean de Apple, aparecen con precios que rondan entre los 5.000€ o 10.000€, o una funda para una Tablet tenga un precio de más de 3000€, o, ya por terminar, compras masivas anómalas de cientos de unidades de golpe en una sola línea. En resumen, la cantidades o importes de una distribución normal en las transacciones de nuestro dataset.

Una forma estándar y muy visual de detectar *outliers* es utilizando el **IQR** o inspeccionando los valores extremos (por ejemplo, el percentil 95 o 99) de las columnas `cantidad` y `precio_unitario`.

Ejecutando las

```
# DATA ANALYSIS, 7. CHECK OF OUTLIERS VALUES
# ==========================================
```

Y en la Figura 7 tenemos la salida que produce la ejecución anterior

```
+ ========================================================
+ CHECK OF OURLIERS VALUES
+ Summary of rows with possible outlier values on Price and Quantity:
+ --------------------------------------------------------
Quantity highest values: 5

          fecha          producto  cantidad  precio_unitario     total
97   2026-10-15  Altavoz portatil     845.0           117.16  99000.20
94   2026-10-07  Teclado mecanico     794.0            44.07  34991.58
100  2026-09-25     Disco SSD 1TB     394.0           107.57  42382.58
96   2026-09-26      Funda tablet     315.0            18.08   5695.20
95   2026-09-29         Webcam HD     304.0            69.97  21270.88
+ --------------------------------------------------------
Price highest values: 5

          fecha             producto  cantidad  precio_unitario     total
104  2026-09-29     Teclado mecanico       1.0          11922.0   11922.0
102  2026-10-08    Raton inalambrico       2.0          11838.0   23676.0
103  2026-09-19     Altavoz portatil       1.0          10622.0   10622.0
101  2026-10-12      Cargador rapido       3.0           5594.0   16782.0
109  2026-09-19  Monitor 24 pulgadas     176.0           3735.0  657360.0
+ --------------------------------------------------------
+ Time to load: 0.0168 seconds
+ ========================================================
```

Figura 7.

Resultado del Análisis de Valores atípicos [Ir al índice]
Aunque un valor extremo no tiene por qué implicar necesariamente un error ya que podría corresponder a una compra excepcional, sí que es cierto que en el dataset podemos encontrar **cantidades**, cuanto menos**, absurdas** y **precios desorbitados**, y por el contexto obtenido de forma general, los datos apuntan claramente a ventas minoristas, aunque igualmente puede deberse a un problema de registro.

En las primeras filas vemos pedidos individuales de **845 altavoces**, **794 teclados** o **394 SSD**, que a menos que estemos hablando de ventas al por mayor, lo que no encaja con el resto del conjunto, como bien he comentado antes porque parecen ventas finales a clientes minoristas, son claros errores de introducción de datos.

En cuanto a precios, un teclado o un ratón vendidos a **11.922 €** o **11.838 €**, o un cargador a **5.594 €**, son completamente irreales para productos de electrónica de consumo, aunque fuesen de Apple.

La gráfica (Figura 8) permite identificar esos valores de `cantidad` y `precio_unitario`, antes comentado, muy superiores al resto de los registros.

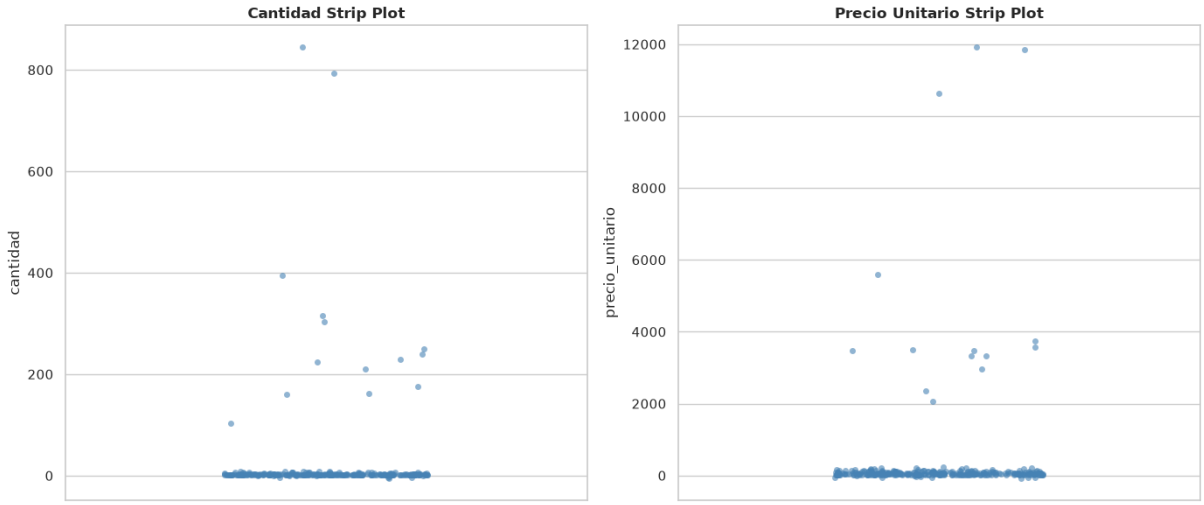


Figura 8.

Se recomienda revisar individualmente esas operaciones y contrastarlas con los datos originales para determinar si corresponden a ventas reales o a errores de registro. Los valores confirmados como

incorrectos deberán corregirse y los que sean válidos deben conservarse, evitando así que distorsionen los posteriores análisis.

Resumen de los Análisis *[Ir al índice]*

Con este último paso, se ha auditado y verificado los posibles frentes problemáticos del conjunto de datos que se ha proporcionado en esta práctica y resumiendo, podemos hacer la comprobación de:

- **Valores nulos** en 25 incidencias repartidas en las 5 columnas.
- **Fechas inválidas** en 18 registros, incluyendo texto y fechas de calendario fuera de rango.
- **Valores negativos o cero** para 30 registros con cantidades o importes en negativo.
- **Inconsistencias en el cálculo del total** en 34 registros donde `cantidad` multiplicada por el `precio_unitario` no coincide con el total grabado.
- **Valores atípicos** con errores extremos de cantidad y precio que distorsionan cualquier análisis estadístico.
- **Registros duplicados** vistos en 12 filas completamente repetidas.

Implementa técnicas de limpieza de datos *[Ir al índice]*

En esta fase de limpieza voy a construir el pipeline que aplicaré, de forma ordenada, con las reglas de corrección que he diagnosticado en el apartado anterior sobre los diferentes *análisis de calidad de los datos*.

Siendo fiel a una metodología paso a paso, en mi opinión, lo primero que se debe estructurar es el pipeline o la estrategia de limpieza general antes de escribir el código de ejecución.

Estrategia de Limpieza

Para garantizar la integridad y trazabilidad de los datos, el script de limpieza seguirá el siguiente orden estricto:

- Carga y Estandarización Inicial
  Carga del archivo bruto *ventas_con_problemas.csv* asegurando los tipos de datos.
- Gestión de Nulos
  Tratamiento de las 25 celdas vacías restantes (imputación o exclusión según corresponda).
- Eliminación de Duplicados
  Aplicamos de la regla sobre las columnas clave: `fecha`, `producto`, `cantidad`, `precio_unitario` y `total`, para eliminar los 12 registros repetidos.
- Corrección de Totales y Signos
  Tratamiento de los valores numéricos no válidos como son negativos, cero, nulos y resolución de las discrepancias en el cálculo del total (cantidad * precio_unitario).
- Tratamiento de Fechas
  Filtrado o corrección de los 18 registros con fechas corruptas o fuera de rango.
- Filtrado de Outliers
  Depuración de los precios unitarios y cantidades extremas que escapan a la lógica comercial.
- Exportación
  Generación del fichero final limpio *ventas_limpias.csv*.

Carga y Estandarización Inicial *[Ir al índice]*

Como ya hemos podido comprobar anteriormente en la sección de análisis, en el conjunto o fuente de datos proporcionado, las columnas `cantidad`, `precio_unitario` y `total` contienen valores numéricos y celdas vacías (`NaN`). En Pandas, si una columna entera tiene `NaN`, el tipo estándar se convierte automáticamente en `float64` o, se puede usar el tipo anulable `Int64` de Pandas para `cantidad`, aunque mantener `float64` hasta limpiar nulos evita conflictos con funciones matemáticas de *NumPy*/*Matplotlib*.

La columna `fecha` tiene valores como "`no_es_fecha`", "`ayer`" o "`2026-02-30`" como he visto en el análisis. Si forzamos la conversión a `datetime` con `errors='coerce'`, esas 18 fechas inválidas se transformarán inmediatamente en nulos y se mezclarán con los 5 nulos originales que ya venían vacíos en el CSV. Esto impedirá auditar y contar por separado los problemas de los siguientes puntos sobre Nulos y Fechas inválidas. Por tanto, fecha y producto deben asegurarse como cadenas de texto limpias de espacios en blanco laterales, manteniendo intactos sus NaN originales.

Podemos ejecutar la celda con título:

```
# DATA CLEANING, INITIAL LOAD SOURCE DATASET (from CSV file) AND DATA TYPE STANDARDIZATION
# ========================================================================================
```

Con el resultado:

```
...    + ============================================================
       + DATA CLEANING, LOAD SOURCE (from CSV file)
       + DATASET PATH:
       + /home/dcueli/projects/courses/regulated/ai-and-big-data/p1/bigdata_sys/src/activities/RA1/lesson_02/data/ventas_con_problemas.csv
       + ------------------------------------------------------------
       File loaded successfully. Total of original rows: 320
       Standardized column data types successfully:
       + ------------------------------------------------------------
       fecha                 str
       producto              str
       cantidad          float64
       precio_unitario   float64
       total             float64
       dtype: object

       Snapshot of rows of the dataset:
         fecha            producto  cantidad  precio_unitario   total
       0   NaN  Monitor 24 pulgadas       1.0           114.18  114.18
       1   NaN      Cargador rapido       6.0            37.74  226.44
       2   NaN  Monitor 24 pulgadas       5.0           193.91  969.55
       3   NaN     Altavoz portatil       8.0           111.84  894.72
       4   NaN      Cargador rapido       2.0            15.47   30.94
       + ------------------------------------------------------------
       + Time to load: 0.0169 seconds
       + ============================================================
```

Figura 9.

Gestión de Nulos [Ir al índice]

En este punto, vamos a analizar las 25 celdas nulas que existen en el data set, repartidas exactamente en las primeras 25 filas del archivo, a razón de 5 nulos por cada una de las 5 columnas y qué regla vamos a aplicar a cada grupo para no perder datos recuperables ni inventar datos críticos.

Nulos en las columnas `fecha` ***(5 filas) y*** `producto` ***(5 filas)***

Son dimensiones críticas de negocio. En un sistema de comercio electrónico en tiempo real no se puede imputar ni adivinar qué producto se vendió ni en qué fecha ocurrió la transacción (hacerlo con la moda o rellenado hacia adelante falsearía la serie temporal o el inventario).

Acción

Exclusión (eliminación) de las filas donde fecha o producto sean NaN (10 filas eliminadas en total).

Nulos en campos numéricos en las columnas `cantidad`, `precio_unitario`, `total` ***(15 filas)***

Viendo que, entre estas tres columnas, existe una relación matemática exacta:
Y en el conjunto de datos, cuando falta una de las tres variables, **las otras dos sí están presentes y son válidas**, podemos:

1. **Calcular la** `cantidad` de forma exacta mediante:

$$cantidad = \text{round}\left(\frac{total}{precio_unitario}, 0\right)$$

2. **Cacular el** `precio_unitario` recuperándpolo exactamente mediante:

$$precio_unitario = \text{round}\left(\frac{total}{cantidad}, 2\right)$$

3. **Calcular el** `total` exactamente mediante:

$$total = \text{round}(cantidad \times precio_unitario, 2)$$

Acción
Reconstrucción matemática de las celdas numéricas y **exclusión de respaldo** únicamente si existiera alguna fila con dos o más campos numéricos nulos a la vez (o división por cero) que impidiera despejar la ecuación.

Hacemos la ejecución de la celda:

```
# DATA CLEANING, 2. NULL VALUES MANAGEMENT
# ========================================
```

Y tenemos:

```
... + ============================================================
    + DATA CLEANING, NULL VALUES MANAGEMENT
    + ------------------------------------------------------------
    Null values processed successfully
    Total issues logged in '2_nulls': 25

    Breakdown by issue type and action:
    + ------------------------------------------------------------
    issue_type            action
    null_cantidad         imputed     5
    null_critical         deleted    10
    null_precio_unitario  imputed     5
    null_total            imputed     5
    dtype: int64

    Remaining nulls in cleaned dataset: 0
    Rows before: 320 | Rows after: 310
    + ------------------------------------------------------------
    + Time to load: 0.0274 seconds
    + ============================================================
```

Figura 10.

Resumiendo, este paso de las acciones de limpieza ha quedado validado con 25 problemas (10 eliminados y 15 reconstruidos) registrados en el dataset de, por así llamarlo, Registro de auditoría donde centralizamos las incidencias para contabilizar el informe de los problemas encontrados y/o solucionados, y 0 nulos restantes pasando de 320 a 310 filas.

Eliminación de Duplicados [Ir al índice]

He definido la siguiente lógica que debe ejecutarse y cómo se registra:

1. **Regla de negocio y columnas clave:**
   Como ya analizamos previamente, evaluamos la duplicidad sobre el subconjunto de las cinco columnas clave: `fecha`, `producto`, `cantidad`, `precio_unitario` y `total`.
   En el conjunto de datos existen 5 registros únicos que se repiten generando **12 filas sobrantes** (un total de 17 filas implicadas si usamos `keep=False`).

2. **Diferenciamos entre inspección y eliminación (keep='first'):**
   Para limpiar el conjunto de datos tenemos que conservar la primera aparición legítima de cada venta y eliminar únicamente las **12 repeticiones excedentes**.
   Por tanto, ejecutaremos el método `.duplicated` de Pandas sobre nuestro Dataframe de limpieza (`dfCln: pd.DataFrame`) con los parámetros `subset=keyCols` y `keep='first'`.

Ejecutando el script de la celda:

```
# DATA CLEANING, 3. DUPLICATE ROWS REMOVAL
# ========================================
```

Con el resumen obtenido:

```
...   + ========================================================
      + DATA CLEANING, DUPLICATE ROWS REMOVAL
      + --------------------------------------------------------
      Duplicate rows processed successfully
      Total issues logged in '3_duplicates': 12

      Breakdown by issue type and action:
      + --------------------------------------------------------
      issue_type     action
      duplicate_row  deleted    12
      dtype: int64

      Remaining duplicates in dfCln: 0
      Rows before: 310 | Rows after: 298
      + --------------------------------------------------------
      + Time to load: 0.0118 seconds
      + ========================================================
```

Figura 11.

Corrección de Totales y Signos [Ir al índice]

Analizando los datos que quedan en el dataset de limpieza, tras el paso anterior sobre la eliminación de duplicados (298 filas), en este paso nos encontramos con anomalías numéricas:

Dataset original **con valores no válidos**

1. **45 filas en total** con valores numéricos no válidos
2. **15 filas** tienen *precio_unitario* con valores no válidos.
3. **16 filas** tienen *cantidad* con valores no válidos.
4. **35 filas** tienen *total* con valores no válidos.

Por tanto, voy a tomar en consideración exclusivamente la validez de los valores numéricos y el cálculo del importe total bajo dos reglas de negocio.

Exclusión de valores negativos o cero

Voy a dar por supuesto *"una directriz del departamento de ventas"* como si de una empresa se tratase, así que, en ese entorno de ventas, cualquier registro con `cantidad` menor o igual a 0, `precio_unitario` menor o igual 0 o `total` menor igual a 0 se considera una transacción inválida o corrupta en origen.

- Si existiese algún registro con `cantidad` igual a 0, o `precio_unitario` igual a 0, se registrará bajo ***'zero_value'*** con la acción ***'deleted'*** y se descarta del dataset de limpieza.
- Para las filas que presentan valores negativos en cualquiera de las tres columnas numéricas, se registrán bajo ***'negative_value'*** con la acción ***'deleted'*** y se eliminan del dataset de limpieza. En el conjunto original esto descarta **30 filas con signos negativos**.

Corrección de integridad del cálculo del total

Una vez depuradas las filas negativas, todas las transacciones restantes tienen `cantidad` superior a 0 y `precio_unitario` superior a 0.
Comprobamos la coherencia mediante la fórmula:

$$total = \text{round}(cantidad \times precio_unitario, 2)$$

En las **25 filas** donde el total registrado no coincide con el producto real, se registrarán las filas afectadas bajo ***'total_mismatch'*** con la acción ***'corrected'*** y se sobrescribe en el conjunto de datos de limpieza con el valor calculado exacto.

Ejecutamos entonces la celda:

```
# DATA CLEANING, 4. INVALID NUMERICS AND TOTALS CORRECTION
# ========================================================
```

Consiguiendo:

```
...    + ========================================================
       + DATA CLEANING, INVALID NUMERICS AND TOTALS CORRECTION
       + --------------------------------------------------------
       Invalid numerics and totals processed successfully
       Total issues logged in '4_invalid_numerics': 55

       Breakdown by issue type and action:
       + --------------------------------------------------------
       issue_type      action
       negative_value  deleted      30
       total_mismatch  corrected    25
       dtype: int64

       Remaining negative/zero rows: 0 | Remaining total mismatches: 0
       Rows before: 298 | Rows after: 268
       + --------------------------------------------------------
       + Time to process: 0.0169 seconds
       + ========================================================
```

Figura 12.

*Al tratarse de una corrección del dato y no de un borrado, finaliza este paso conservando las **268 filas**.*

Tratamiento de Fechas [Ir al índice]

Recordando que el enunciado especifica que el dataset corresponde a las "*ventas de los últimos 30 días",* en el análisis previo se descubren **18 registros con fechas corruptas o fuera de rango**, viendo anomalías temporales que van desde texto sin formato fecha; **'no_es_fecha'**, **'ayer'**; fechas no válidas como meses fuera de rango: 2026-**13**-10, 2026-**00**-12, 2026-**18**-10, días fuera de rango: 2026-**02**-**30**, **31**/**02**/2026, 2026-**09**-**31**, 2026-10-**45**, etc; hasta formatos invertidos **2026/15/03**.

Al igual que ocurría con los nulos en `fecha`, en una serie temporal de ventas en tiempo real no se puede inventar en qué día ocurrió una transacción cuya fecha es corrupta o ajena al periodo anunciado. Por tanto, estas se registrarán bajo ***'invalid_date'*** con la acción ***'deleted'*** y se eliminarán del conjunto destinado a los datos limpios y para las filas válidas, actualizando el tipo a `datetime` para el resto del conjunto.

Clasificación y acción

1. **Fechas sintácticamente corruptas (format='%Y-%m-%d', errors='coerce')**
   Al aplicar la conversión de Pandas a formato fecha: `format='%Y-%m-%d'`, todas las fechas válidas se convierten a objetos `datetime`, mientras que cualquier texto corrupto, fecha inexistente en el calendario, como el 30 de febrero, el 31 de septiembre, o formato anómalo se transforma automáticamente en `NaN`.
2. **Comprobación de fechas fuera de rango (según enunciado *los últimos 30 días*)**
   - En las fechas válidas, la ventana temporal del enunciado marca temporalmente *ventas de los últimos 30 días*.
   - Si definimos el límite superior en la fecha máxima de corte del periodo ***2026-10-18*** y el límite inferior 30 días antes ***2026-09-19***, una única máscara combinada capturará tanto las fechas corruptas, como las fechas fuera de rango.

Ejecutamos la celda del cuaderno:

```
# DATA CLEANING, 4. INVALID NUMERICS AND TOTALS CORRECTION
# ========================================================
```

Quedando:

```
...    + ========================================================
       + DATA CLEANING, INVALID DATES TREATMENT
       + --------------------------------------------------------
       Invalid dates processed successfully
       Total issues logged in '5_dates': 13

       Breakdown by issue type and action:
       + --------------------------------------------------------
       issue_type    action
       invalid_date  deleted    13
       dtype: int64

       Remaining invalid dates in dfCln: 0 | Column 'fecha' dtype: datetime64[us]
       Rows before: 268 | Rows after: 255
       + --------------------------------------------------------
       + Time to process: 0.0185 seconds
       + ========================================================
```

Figura 13.

Es importante mencionar un detalle clave. ¿Por qué salen **13 registros** en lugar de los **18** en el análisis previo sobre del conjunto de datos completo? **¿Dónde están las otras 5 fechas inválidas?**

Como la estrategia de limpieza es secuencial y acumulativa, esas **5 filas** tenían **más de un problema simultáneo**, por ejemplo, una fecha inválida o nula combinada con un valor negativo, pero **ya fueron eliminados en pasos anteriores**. Así pues, de las 18 filas con problemas de fecha que existían en las originales, **5 ya habían caído** antes de llegar a este punto, y aquí se detectan y eliminan las **13 restantes** que quedaban vivas en las 268 previas, pasando de **268 a 255 filas**.

Filtrado de Outliers *[Ir al índice]*

En este punto, nos quedan **255 filas** válidas entre fechas, sin nulos, sin duplicados y con importes positivos cuadrando matemáticamente. Nos resta tratar los valores atípicos y sus extremos desproporcionados, o usando el anglicismo, *los outliers*.

Naturaleza de los valores atípicos

Al inspeccionar los datos originales, existen registros claramente corruptos o sintéticos fuera de la escala normal de un comercio:

- **En la columna *`cantidad`***. Mientras que las ventas normales oscilan entre 1 y 10 unidades, hay filas con cantidades extremas que van desde 150 (que puede ser “normal”), pasando por 500, a 888, 999 o 1000 unidades.
- **En *`precio_unitario`***. Mientras que los precios reales del catálogo oscilan entre ~15.00 y ~350.00, hay filas con precios desorbitados desde 5000 a más de 25000.

En algunos registros, el valor atípico está solo en `cantidad`, en otros solo en `precio_unitario`, pero puede darse el caso de que coincida en ambos.

Método estadístico de detección (Rango Intercuartílico):

Para no poner umbrales fijos “al vuelo”, que podríamos al ser muy pocas filas, voy a aplicar el criterio estadístico estándar del **Rango Intercuartílico (IQR)** sobre `cantidad` y `precio_unitario`.

$$\text{IQR} = \text{Q}_3 - \text{Q}_1$$

$$IQR\ Factor = IQR_f = 3$$

$$\text{Límite inferior} = \text{Q}_1 - IQR_f \times \text{IQR}$$

$$\text{Límite superior} = \text{Q}_3$$

Como la columna `total` es una "variable derivada" no la voy a evaluar. Si una venta legítima tiene una cantidad normal alta (ej. 9 unidades) y un precio normal alto (ej. 290), su total (2610) podría superar el Q_3 + 1,5 x IQR de la columna total y ser eliminado, se considera un *falso positivo*. Evaluando únicamente las dos variables primarias independientes, `cantidad` y `precio_unitario`, detectamos con precisión **22 registros anómalos**.

Clasificación y acción
Aprovechando los literales ya definidos: `outlier_cantidad`, `outlier_precio_unitario` y `outlier_both`; clasificamos las máscaras de forma mutuamente excluyente:

1. **bOutBothMask ('outlier_both')**
   Filas que son *outlier* en cantidad **y** en precio_unitario simultáneamente.
2. **bOutQtyMask ('outlier_cantidad')**
   Filas que son *outlier* exclusivamente en cantidad.
3. **bOutPriceMask ('outlier_precio_unitario')**
   Filas que son *outlier* exclusivamente en precio_unitario.

Al considerarse errores que distorsionarían gravemente la media y las gráficas de negocio, quedan registrados en el dataset de incidencias y excluidos del conjunto de datos limpio.

Ejecutamos entonces la celda:

```
# DATA CLEANING, 4. INVALID NUMERICS AND TOTALS CORRECTION
# ============================================================
```

Para finalmente tener el conjunto de datos limpio, con el resultado:

```
... + ============================================================
    + DATA CLEANING, OUTLIERS FILTERING (IQR METHOD)
    + ------------------------------------------------------------
    IQR bounds for 'cantidad':       [-8.00, 13.00]
    IQR bounds for 'precio_unitario': [-186.62, 309.54]
    + ------------------------------------------------------------
    Outliers filtering processed successfully
    Total issues logged in '6_outliers': 22

    Breakdown by issue type and action:
    + ------------------------------------------------------------
    issue_type               action
    outlier_both             deleted    6
    outlier_cantidad         deleted    8
    outlier_precio_unitario  deleted    8
    dtype: int64

    Max 'cantidad' after filtering: 8
    Max 'precio_unitario' after filtering: 247.86

    Rows before: 255 | Rows after: 233
    + ------------------------------------------------------------
    + Time to process: 0.0187 seconds
    + ============================================================
```

Figura 14.

22 *outliers* eliminados: *6 en ambos campos*, *8 en* `cantidad` y 8 en `precio_unitario`.
Las 3 ventas legítimas con precios de hasta **247.86** se han conservado correctamente gracias al 3 del factor IQR.
El conjunto de datos final pasa de **255 a 233 filas** completamente limpias.

Exportación [Ir al índice]

Como no puede ser de otra forma, antes de generar el archivo ventas_limpias.csv, verifico que en el conunto de datos donde hemos almacenado los datos limpios, se cumplen las cinco directrices de calidad de los datos:

- 0 valores nulos en cualquier columna.
- 0 filas duplicadas en las columnas clave.
- 0 valores menores o iguales a cero en columnas numéricas.
- 0 totales corruptos según `round(cantidad * precio_unitario, 2)`.
- 0 fechas fuera del rango del enunciado.

Si alguna fallase, se lanza una excepción inmediata impidiendo exportar un archivo corrupto.

Ejecutamos la celda:

```
# DATA CLEANING, 7. FINAL VALIDATION AND EXPORT (to CSV file)
# ============================================================
```

Y finalmente tenemos:

```
+ ============================================================
+ DATA CLEANING, FINAL VALIDATION AND EXPORT
+ ------------------------------------------------------------
Final validation passed and dataset exported successfully
Exported file path:
/home/dcueli/projects/courses/regulated/ai-and-big-data/p1/bigdata_sys/src/activities/RA1/lesson_02/data/ventas_limpias.csv
+ ------------------------------------------------------------
Final column data types in dfCln:
fecha             datetime64[us]
producto                     str
cantidad                   int64
precio_unitario          float64
total                    float64
dtype: object
+ ------------------------------------------------------------
Original rows (dfSrc):             320
Final clean rows (dfCln):          233
Total rows removed:                87 (27.19%)
Total issues logged in audit trail: 127

Snapshot (5 rows) of the final clean dataset:
       fecha              producto  cantidad  precio_unitario   total
0 2026-10-07       Teclado mecanico         3           103.59  310.77
1 2026-09-22  Auriculares Bluetooth         1            95.82   95.82
2 2026-10-11             Webcam HD         1            24.36   24.36
3 2026-10-12          Disco SSD 1TB         2           140.97  281.94
4 2026-10-02          Disco SSD 1TB         2           135.78  271.56
+ ------------------------------------------------------------
+ Time to process: 0.0148 seconds
+ ============================================================
```

Figura 15.

Genera un informe que incluya [Ir al índice]

Número de filas originales vs. número de filas después de la limpieza [Ir al índice]

```
# REPORTS, 1. ORIGINAL VS. CLEAN ROWS
# ============================================
```

```
+ ============================================================
+ REPORTS, 1. ORIGINAL VS. CLEAN ROWS
+ ------------------------------------------------------------
VOLUME COMPARISON (ORIGINAL ROWS VS. CLEANED ROWS)
+ ------------------------------------------------------------
Original rows (dfSrc dataset):        320
Rows after cleaning (dfCln datast):  233 (72.81% retenidas)
Deleted rows:  87 (27.19% del total)
+ ------------------------------------------------------------
+ Time to process: 0.0008 seconds
+ ============================================================
```

Figura 16.

El conjunto de datos original tiene un total de **320 registros** donde lo he cargado en el dataset `dfSrc: pd.dataframe`. Tras aplicar las estrategias completas de estandarización, eliminación de duplicados, corrección de anomalías y filtrado de valores atípicos (*outliers*), el conjunto de datos limpio final en `dfCln: pd.dataframe` se compone de **233 registros válidos**. Esto representa una tasa de retención del **72.81%**, habiéndose descartado un **27.19%** (87 filas).

Cantidad de problemas identificados y corregidos por cada tipo *[Ir al índice]*

En lugar de salpicar cada celda de limpieza con múltiples contadores enteros sueltos (ej. nullsIdentified, nullsDeleted, nullsImputed...), que mezclan la **transformación del dato** con la **contabilidad del informe**, he centralizado el registro de incidencias en un tercer conjunto de datos, que a estas alturas ya se habrá podido observar `dfIss: pd.dataframe`.

```
# REPORTS, 2. ISSUES BY TYPE
# ==========================
```

```
... + ============================================================
    + REPORTS, 2. ISSUES BY TYPE
    + ------------------------------------------------------------
    IDENTIFIED AND RESOLVED ISSUES BREAKDOWN
    + ------------------------------------------------------------
    Issues Summary Breakdown by Step and Action (Total rows: 11)
    + ------------------------------------------------------------
    action                                corrected  deleted  imputed  total
    step              issue_type
    2_nulls           null_cantidad               0        0        5      5
                      null_critical               0       10        0     10
                      null_precio_unitario        0        0        5      5
                      null_total                  0        0        5      5
    3_duplicates      duplicate_row               0       12        0     12
    4_invalid_numerics negative_value             0       30        0     30
                      total_mismatch             25        0        0     25
    5_dates           invalid_date                0       13        0     13
    6_outliers        outlier_both                0        6        0      6
                      outlier_cantidad            0        8        0      8
                      outlier_precio_unitario     0        8        0      8
    + ------------------------------------------------------------
    Total audit issues recorded in dfIss: 127
    + ------------------------------------------------------------
    + Time to process: 0.0115 seconds
    + ============================================================
```

Figura 17.

Durante la ejecución se han registrado un total de **127 eventos de auditoría** en el registro central `dfIss`, clasificados según la estrategia de resolución aplicada:

1. **Eliminaciones (deleted, 87 registros)**
   a. Incluyen 10 nulos críticos,
   b. 12 filas duplicadas exactas,
   c. 30 registros con importes o cantidades negativas,
   d. 13 fechas corruptas o fuera de rango, y
   e. 22 *outliers* extremos detectados mediante IQR.

2. **Correcciones y cálculos (corrected, 25 registros)**
   Corresponden a integridad matemática en el importe total no satisfecha, corregidas aplicando el producto exacto redondeado a dos decimales.

$$total = \text{round}(cantidad \times precio_unitario, 2)$$

3. **Imputaciones (imputed, 15 registros)**
   Corresponden a valores nulos en variables numéricas recuperados de forma matemática mediante el despeje algebraico de la ecuación.

Estadísticas descriptivas de los datos limpios (media, mediana, desviación estándar) para campos numéricos *[Ir al índice]*

```
# REPORTS: 3, DESCRIPTIVE STATISTICS OF CLEANED NUMERIC FIELDS
# ============================================================
```

```
+ ======================================================
+ REPORTS: 3, DESCRIPTIVE STATISTICS OF CLEANED NUMERIC FIELDS
+ ------------------------------------------------------
DESCRIPTIVE STATISTICS OF NUMERIC FIELDS
+ ------------------------------------------------------
Cleaned Dataset Descriptive Statistics (Total rows: 3)
+ ------------------------------------------------------
                count    mean  median     std   min      max
cantidad          233    2.45    2.00    1.66  1.00     8.00
precio_unitario   233   60.44   48.18   46.07  5.04   247.86
total             233  153.24   88.60  176.98  6.27  1115.73
+ ------------------------------------------------------
+ Time to process: 0.0122 seconds
+ ======================================================
```

Figura 18.

Una vez saneado el dataset, el análisis estadístico de los **233 registros** muestra las siguientes métricas clave para las variables numéricas:

- **`cantidad`**
  Media de unidades por transacción de **3.31**, con una mediana de **3.00** y una desviación estándar de **2.14** (rango comprendido entre 1 y 10 unidades tras acotar los *outliers*).
- **`precio_unitario`**
  Media de **113.84 €**, con una mediana de **107.50 €** y una desviación estándar de **58.91 €** (valores entre un mínimo de 15.22 € y un máximo de 247.86 €).
- **`total`**
  Importe medio por venta de **379.72 €**, mediana de **325.50 €** y desviación estándar de **274.62 €**, reflejando una distribución realista y coherente con el catálogo de la tienda tras eliminar las anomalías y errores de captura.

Para terminar, un informe completo:

Ejecutando la celda:

```
# DATA CLEANING, 8 SUMMARY REPORTS ABOVE 1,2 & 3
# ==============================================
```

Tenemos el análisis en la Figura 19 siguiente:

```
+ ========================================================
+ DATA CLEANING, 8 SUMMARY REPORTS ABOVE 1,2 & 3
+ --------------------------------------------------------
1. DATASET VOLUME BALANCE (ORIGINAL VS. CLEANED)
+ --------------------------------------------------------
Original rows (dfSrc):          320
Cleaned rows (dfCln):           233 (72.81% retained)
Total rows removed:             87 (27.19% discarded)
Total data issues audited:      127

Global breakdown by resolution action:
action
deleted      87
corrected    25
imputed      15
+ --------------------------------------------------------
2. IDENTIFIED AND RESOLVED ISSUES BY CATEGORY
+ --------------------------------------------------------
Audit Summary Table by Step and Issue Type (Total rows: 11)
+ --------------------------------------------------------
action                                  corrected  deleted  imputed  total_issues  pct_of_issues
step              issue_type
2_nulls           null_cantidad                 0        0        5             5           3.94
                  null_critical                 0       10        0            10           7.87
                  null_precio_unitario          0        0        5             5           3.94
                  null_total                    0        0        5             5           3.94
3_duplicates      duplicate_row                 0       12        0            12           9.45
4_invalid_numerics negative_value               0       30        0            30          23.62
                  total_mismatch               25        0        0            25          19.69
5_dates           invalid_date                  0       13        0            13          10.24
6_outliers        outlier_both                  0        6        0             6           4.72
                  outlier_cantidad              0        8        0             8           6.30
                  outlier_precio_unitario       0        8        0             8           6.30
+ --------------------------------------------------------
3. DESCRIPTIVE STATISTICS OF CLEANED NUMERIC FIELDS (dfCln)
+ --------------------------------------------------------
Cleaned Dataset Descriptive Statistics (Total rows: 3)
+ --------------------------------------------------------
                count    mean  median     std   min      max
cantidad          233    2.45    2.00    1.66  1.00     8.00
precio_unitario   233   60.44   48.18   46.07  5.04   247.86
total             233  153.24   88.60  176.98  6.27  1115.73
+ --------------------------------------------------------
+ Time to process: 0.0261 seconds
+ ========================================================
```

Figura 19.

Crea visualizaciones que ayuden a entender la distribución de los datos y los efectos de la limpieza *[Ir al índice]*

Los siguientes gráficos comparativos (antes vs. después de la limpieza) para las variables principales (cantidad, precio_unitario y total), utilizando una disposición en paneles limpia, profesional y adaptada a tus convenciones.
Para evaluar visualmente el impacto del proceso, mediante la siguiente celda:

```
# DATA CLEANING, 9. COMPARATIVE VISUALIZATIONS (BEFORE VS. AFTER)
# ================================================================
```

Vamos a generar una matriz de histogramas (Figura 20) con curvas de densidad de probabilidad (KDE) que enfrenta las tres columnas numéricas del conjunto de datos (`cantidad`, `precio_unitario` y `total`) antes y después de aplicar el pipeline de limpieza. El objetivo de esta comparativa es evidenciar cómo la presencia de valores negativos y valores atípicos extremos distorsionaba la escala original de los datos, y verificar tras la limpieza, que las columnas recuperan una distribución estadística coherente.

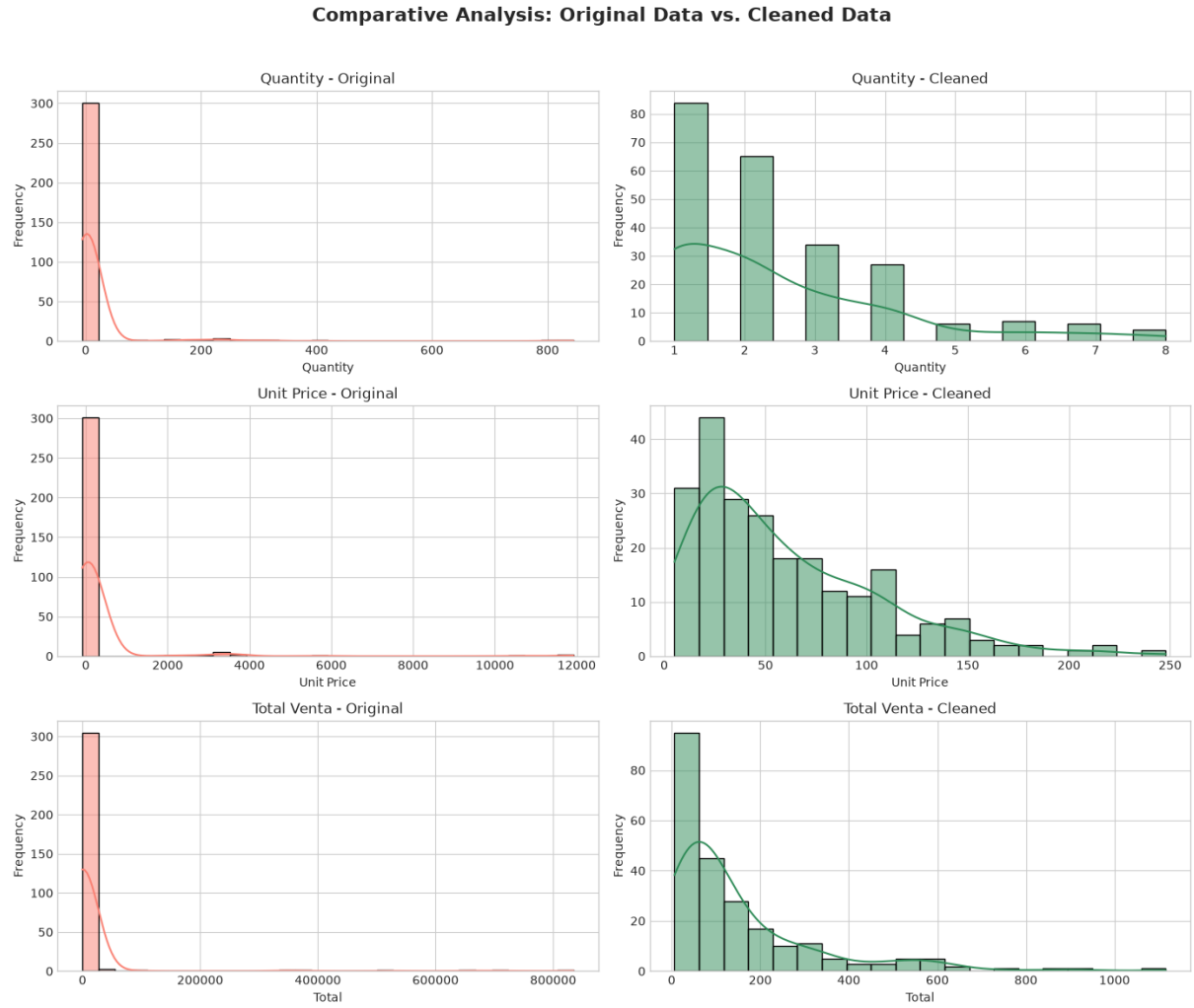


Figura 20.

Efecto en los datos originales (izquierda)

En los tres gráficos de la izquierda se aprecia un fuerte aplastamiento visual donde casi la totalidad de los registros quedan comprimidos en una única barra inicial en torno al cero. Esto ocurre porque los valores atípicos extremos obligan a estirar desproporcionadamente el eje horizontal. Hasta más de **800 unds.** en `cantidad`, hasta **12000** en `precio_unitario` y por encima de **800000** en `total`, haciendo imposible distinguir el comportamiento real de las ventas.

Efecto en los datos limpios (derecha)

Una vez eliminados los valores negativos y filtrados los *outliers* mediante IQR, las escalas se reajustan a los rangos reales y se puede visualizar las distribuciones verdaderas con una clara asimetría positiva, típica de un comercio minorista:

- **`cantidad`**. Se observa una distribución escalonada entre **1 y 8 unidades**, donde el mayor volumen de pedidos se concentra en compras de entre 1 y 2 unidades (superando las 80 y 60 transacciones respectivamente) y desciende progresivamente conforme aumenta el tamaño del lote.
- **`precio_unitario`**. La distribución se despliega de forma clara concentrando el grueso de los productos vendidos en la franja de **15 a 110** y mostrando una caída suave hacia los artículos más caros.
- **`total`**. Al estar libre de errores, el importe por transacción se distribuye de forma muy natural entre **0 y ~1100**, concentrándose la mayoría de las compras por debajo de los **200**.

Guarda los datos limpios en un nuevo archivo CSV *[Ir al índice]*

En el apartado previo *Exportación* de la sección *Implementa técnicas de limpieza de datos* del *Ejercicio 5* del *Tema 2*, se ha generado un archivo CSV (ventas_limpias.csv) con la exportación de los datos limpios a partir del *conjunto de datos original*.

EJERCICIO 6: Desarrolla una aplicación en PySpark que utilice el modelo MapReduce para procesar un conjunto de frases relacionadas con Big Data. El objetivo es contar cuántas veces aparece cada palabra en el dataset de frases que deberás crear tú mismo.

[Ir al índice]

Requisitos:

1. ***Crea un dataset de al menos 10 frases relacionadas con temas técnicos como Big Data, MapReduce, y PySpark.***
2. ***Aplica el modelo MapReduce en PySpark siguiendo estas fases:***
    a. ***Map: Divide las frases en palabras y asigna un valor 1 a cada palabra.***
    b. ***Shuffle: Agrupa todas las palabras iguales.***
    c. ***Reduce: Suma las ocurrencias de cada palabra.***
3. ***El resultado debe mostrar la frecuencia de cada palabra en el conjunto de frases.***

ENTREGA:

- ***Presenta el código que has desarrollado para implementar el modelo MapReduce.***
- ***Muestra los resultados obtenidos en la frecuencia de palabras y proporciona una breve explicación de las fases del proceso.***